UCCD2063 Artificial Intelligence Techniques Practical Assignment

June - Sept 2026

1. General Instructions

The general guideline for this assignment is as follows:

- The **total mark** for the **assignment is 100 and contributes 30% to the total grade**.
- This is a group assignment. Each group consists of **ONLY FOUR** students.
- Submit ONE Zip file containing the report (PDF), code (.ipynb), Godot project files/scenes, LLM model prompt, and other related materials (if any) to WBLE before/on the deadline.
- The **deadline** for the **submission** of reports is on **August 30th (Sunday), 5 pm**. Late submissions may be subject to a penalty of up to 50% of the total assignment mark.
- Evidence of plagiarism will be taken seriously, and University regulations will be applied fully to such cases, in addition to ZERO marks being awarded to all parties involved.

2. Introduction

In this assignment, you are required to implement reinforcement learning (RL) models that learn to play in a game environment developed using Godot. The selected game concept is a 2D Platform Collector Game in which an agent controls a character to move across platforms, jump between them, avoid enemies, collect rewards, and complete the level.

The objective of this assignment is to train RL agents that can improve their gameplay performance through repeated interaction with the game environment. The agent should learn suitable actions and progress through the platform map based on the current game state to maximize the total reward.

The purpose of the assignment is to allow students to understand how RL works in a game-based problem. Students will learn to define states and rewards, implement an RL technique, train the agent, tune learning parameters, and evaluate the agent's performance.

Unlike supervised learning, this assignment does not rely on a fixed labeled dataset. Instead, the agent learns from trial and error by interacting with the 2D Platform Game scenes. The learning process is based on states, actions, rewards, exploration, exploitation, and policy improvement.

3. Requirements

In this assignment, each student is given one different Godot game scene and is responsible for selecting and implementing one RL technique for that scene. All scenes are based on the same 2D Platform Collector Game concept, but the design differs in platform layout, route design, reward placement, and enemy placement.

3.1 Godot Game Environment

The game concept is based on a platform map featuring a player character, platforms, rewards, enemies, boundaries, and unsafe fall areas. The environment should allow the agent to interact with the Godot scene by selecting an action. After each action, the scene should return the next state, reward, game status, and score. This is important because RL depends on the interaction between the agent and the environment.

3.2 Allocation and Reinforcement Learning Technique Selection

Each student is responsible for one Godot scene and one RL model. Students may select any suitable RL technique, such as Q-Learning, SARSA, Expected SARSA, DQN, Double DQN, PPO, or another method.

3.3 State, Action, and Reward Design

Example state information may include:

- Player x-position and y-position
- Whether the player is on the ground
- Distance and direction to the nearest reward
- Whether there is an enemy nearby
- Distance to the nearest enemy
- Whether there is a platform or safe landing area nearby
- Whether there is a gap or a falling area nearby
- Distance to the goal or end of level

The action space should define the possible actions that the agent can take. For example:

- Do nothing
- Move right
- Move left
- Jump
- Move right and jump
- Move left and jump

The reward function should guide the agent to learn good gameplay behavior. Example reward design:

- Positive reward for collecting a reward, moving closer to a reward, or progressing through the level
- Large positive reward for reaching the goal or completing the level
- Negative reward for hitting an enemy, falling from a platform, or staying still too long
- Small negative reward for each time step to encourage faster completion

Students should explain why the selected state representation, action space, and reward function are suitable for the scenario and their chosen RL technique.

3.4 Experiment

The experiment should not only train the agents using default settings. Each student should test and analyze their own RL technique using different settings in their assigned Godot scene design. Each student should record the training performance of their own agent, and the evaluation may include:

- Total reward per episode
- Average score after training
- Number of rewards collected, snail enemies avoided
- Distance travelled
- Training time
- Learning curve

3.5 Coding

Students must use Godot for the game scenes and Python 3.6 or later for the RL implementation. Python packages such as NumPy, Matplotlib, TensorFlow, PyTorch, or other suitable libraries may be used for hyperparameter tuning, result analysis, and visualization.

The code should be structured clearly into several parts, for example:

- Godot scene file for the assigned game level
- State representation and reward function
- RL agent
- Training process
- Testing process
- Result visualization

The code must be well-commented so that each part of the implementation is clear. Students must provide clear instructions for running the RL agent and reproducing the results shown in the report.

3.6 Report

The report should include the following, but not limited to, for each student's assigned scene design and selected RL technique:

- Explanation of the selected RL technique
- Training setup and parameter settings
- Training and testing results for the selected technique and scene design

- Discussion of the agent's performance

The maximum number of pages of the report is 20, excluding the cover page, reference, and appendix. Each member is allocated approximately 5 pages.

4. Format Report

Font: Times New Roman, 10 points, 1.0-line spacing.

Citing references in text: refer to the citation style "IEEE Style Referencing".

5. Marking Criteria

The grading of your assignment will be based on the following criteria:

Criteria	Weight
Explanation of reinforcement learning techniques	15%
Training setup and parameter settings	15%
Training and testing results for each technique	20%
Discussion of the agent's performance	20%
Presentation	10%
LLM prompt generation	5%
Report Documentation	5%
Effort (e.g., in-depth analysis, self-learned learning algorithm)	5%
Coding style (variable naming, comments, formatting, etc.)	5%
Total	**100%**

Rubric

Criteria	Excellent	Good	Satisfactory	Marginal	Poor
Explanation of reinforcement learning techniques (15%)	Excellent explanation of the selected reinforcement learning technique. Clearly explains the algorithm concept, update process, policy, exploration and exploitation, and suitability for the assigned Godot scene.	Good explanation of the selected reinforcement learning technique. The most important concepts are explained clearly, with only minor details missing.	Satisfactory explanation of the technique. The basic concept is explained, but some important details are limited or unclear.	Brief or limited explanation of the technique. Shows weak understanding of how the algorithm works.	Poor explanation of the technique. The algorithm is unclear, incorrect, or not properly related to reinforcement learning.
Training setup and parameter settings (15%)	Excellent description of the training setup. Clearly explains the Godot scene, state, action, reward function, number of episodes, learning rate, discount factor, epsilon, epsilon decay, and other parameters.	Good description of the training setup. Most parameters are included and explained, with only minor details missing.	Satisfactory description of the training setup. Basic parameters are included, but the explanation is not detailed enough.	Limited description of the training setup. Several important parameters are missing or not justified.	Poor description of the training setup. Parameters are missing, unclear, or not related to the experiment.
Training and testing results for each technique (20%)	Excellent presentation of training and testing results. Results are shown using suitable tables, graphs, and screenshots. Metrics such as reward, score, apples collected, enemies avoided, completion rate, survival time, or training time are clearly reported.	Good presentation of results with suitable tables or graphs. Most important metrics are included, with minor details missing.	Satisfactory presentation of results. Some results are shown, but the analysis or visualization is limited.	Limited results are presented. Results are unclear, incomplete, or difficult to compare.	Poor or missing results. No clear evidence that the agent was trained and tested properly.
Discussion of the agent's performance (20%)	Excellent discussion of the agent's performance. Clearly explains whether the agent improved, compares strengths and weaknesses, analyzes failure cases, and relates the results to the selected RL technique and Godot scene.	Good discussion of performance. Provides a reasonable explanation of results and some comparison or failure analysis.	Satisfactory discussion. A basic explanation of performance is provided, but it lacks depth.	Limited discussion. Mostly describes results without proper analysis.	Poor discussion. No meaningful explanation of the agent's performance or learning behavior.
Presentation (10%)	Excellent presentation. Clear explanation, well-organized slides, good visual support, confident delivery, good time management, and strong ability to answer questions.	Good presentation. Mostly clear and organized with minor issues in explanation, slides, or timing.	Satisfactory presentation. Main ideas are presented, but the explanation or slide organization could be improved.	Marginal presentation. Explanation is unclear, slides are weak, or time management is poor.	Poor presentation. Difficult to understand, unorganized, or unable to answer basic questions.
LLM prompt generation (5%)	Excellent documentation of LLM prompts. Shows relevant prompts, follow-up prompts, generated outputs, and explains how the output was modified, verified, or applied.	Good documentation of LLM prompts. Shows useful prompts and some explanation of how the output was used.	Satisfactory documentation. Some prompts are shown, but explanations of usage or verification are limited.	Limited documentation. Prompts are shown briefly but not clearly connected to the assignment work.	Poor or missing LLM prompt documentation. No clear evidence of how LLM tools were used responsibly.
Report documentation (5%)	Excellent report documentation. The report is complete, well-structured,	Good report documentation with minor formatting or organization issues.	Satisfactory documentation. The report is understandable but has some missing	Marginal documentation. The report structure is weak, or several required	Poor documentation. The report is poorly organized, incomplete, or difficult to

Criteria	Excellent	Good	Satisfactory	Marginal	Poor
	properly formatted, and includes suitable tables, figures, captions, and references.		formatting, captions, or references.	parts are missing.	follow.
Effort, in-depth analysis, and self-learnt learning algorithm (5%)	Excellent effort shown. Student demonstrates self-learning, deeper analysis, meaningful comparison, parameter tuning, or advanced improvement beyond basic implementation.	Good effort shown. Student attempts additional analysis, tuning, or self-learnt implementation.	Satisfactory effort. The basic implementation is complete, with some analysis.	Limited effort. Minimal implementation and little evidence of self-learning or deeper analysis.	Poor effort. Very little contribution, analysis, or understanding shown.
Coding style, variable naming, comments, and formatting (5%)	Code is very clean, well-organized, well-commented, and easy to read. Variable names are meaningful, and the notebook or Godot scripts can be run easily.	The code is clean and organized, with minor issues. Comments and variable names are mostly clear.	Code is reasonably readable but may contain inconsistent naming, formatting, or limited comments.	The code is difficult to follow in some parts. Poor formatting, unclear variables, or insufficient comments.	The code is very poor, unorganized, difficult to understand, or not runnable.